Predicting Ski Jumps Using State-Space Model

Neca Camlek*
Univerza v Ljubljani
Ljubljana, Slovenia

Živa Hegler*
Univerza v Ljubljani
Ljubljana, Slovenia

Jakob Jelenčič
Jožef Stefan Institute
Ljubljana, Slovenia
jakob.jelencic@ijs.si

Marko Grobelnik
Jožef Stefan Institute
Ljubljana, Slovenia
marko.grobelnik@ijs.si

Dunja Mladenić
Jožef Stefan Institute
Ljubljana, Slovenia
dunja.mladenic@ijs.si

Abstract

Ski jumping performance is shaped by both athlete technique and environmental conditions, with factors such as wind speed, wind direction, and ski orientation playing a critical role in determining jump trajectories. Accurate modeling of these trajectories is challenging due to dynamic and time-dependent nature of the data. In this work, we introduce a dataset of measured ski jumps and present a state-space modeling framework that captures the evolution of jumps under varying conditions. The model parameters are estimated using a ridge regression approach, enabling us to predict trajectories from initial states and wind sensor inputs. We evaluated the predictive performance of the model through leave-one-out cross-validation and analyzed its stability, showing that the approach can predict realistic trajectories with reasonable accuracy. To complement the modeling results, we developed an interactive web application that allows users to explore both recorded and simulated jumps, adjust environmental factors, and visualize their effects through animations. Together, the dataset, modeling framework, and the web application offer a foundation for further research in ski jump analysis and provide an accessible tool for exploring the influence of external conditions on performance.

Keywords

datasets, state-space model, ski jumping, simulations, least squares

1 Introduction

Ski jumping is a sport strongly influenced by both athletic technique and environmental conditions. Factors such as wind speed, wind direction, and different ski angles affect the trajectory and final distance of a jump, making accurate prediction a challenging problem. While statistical models and simulations have been applied in sports research for some time, many approaches simplify the problem and do not fully capture the dynamic evolution of the jump over time [13, 5, 10, 17].

Recent advances in machine learning have introduced methods capable of modeling temporal systems with greater fidelity. In particular, state-space models provide a mathematical framework for representing hidden internal states that evolve over time in response to external input. This makes them well-suited for modeling ski jumps, where environmental factors determine performance [11].

In this paper, we present a ski jump dataset together with a state-space model trained to predict jump trajectories based on changing environmental conditions. The model is estimated using a least squares approach and demonstrates how inputs such as wind and ramp adjustments influence the resulting jump. Beyond the modeling framework, we also developed an application that allows general users to interact with the data, run simulations, and visualize jump trajectories through animations.

Beyond methodological interest, accurate prediction of ski jumps can improve athlete safety by anticipating risky conditions, support planning of hill design or enlargement, and contribute to fairer competitions through a better understanding of environmental effects.

The remainder of the paper is as follows. Section 2 reviews related work and then section 3 presents the handling of received data. Next, the proposed methodology is described in Section 4. The project results are presented in Section 5. We discuss the results in Section 6 and conclude the paper in Section 7.

2 Related Work

Several approaches have been explored for modeling and predicting ski jump trajectories, ranging from physics-based simulations to data-driven methods.

Fang et al. [5] proposed a state-space approach for reconstructing ski jump trajectories from wearable inertial sensors, using an Extended Rauch-Tung-Striebel (RTS) smoother with state constraints to estimate position and velocity during flight. While their focus was on reconstruction from sensor data rather than prediction from environmental inputs, their use of a state-space formulation for capturing jump dynamics is closely related to our approach.

Schwinn et al. [10] introduced *xLength*, a deep learning framework for predicting the expected jump length shortly after takeoff. Using a dataset of 2523 jumps from 205 athletes across five venues, they evaluated fully connected networks, CNNs, LSTMs, and ResNet architectures, achieving a mean absolute error of 5.3 m when generalizing to new athletes. Their work highlights the potential of sensor-based data for jump length prediction, though it focuses on length estimation from early flight data rather than full trajectory modeling under varying wind conditions.

Zecha et al. [17] presented a convolutional sequence-to-sequence model for predicting multimodal jump dynamics, including trajectory and forces, from athlete pose estimates. This approach demonstrates the use of temporal deep learning models for capturing sequential jump dynamics, but relies on video-based pose input rather than direct environmental sensor data.

In contrast to these works, our approach uses a linear state-space model estimated via ridge regression, which offers greater interpretability and does not require large datasets or deep learning infrastructure. We explicitly incorporate wind sensor measurements as control inputs, allowing the model to simulate how

*Both authors contributed equally to this research.

changing environmental conditions affect the full jump trajectory.

3 Approach to Modeling

This section describes the handling of data, focusing on state-space models and our data processing.

3.1 State-Space Model

State-Space Models (SSMs) are a family of machine learning algorithms designed to capture and predict the behavior of dynamic systems by describing how their inner states change over time. Instead of only looking at past inputs and outputs, SSMs explicitly model the underlying dynamics, making them well-suited for sequential data. In state-space modeling, the objective is to identify the minimal set of system variables required to completely describe the system. These fundamental variables are referred to as the state variables. At any given time, the state of the system can be represented by a state vector, whose components correspond to the values of the respective state variables. SSMs are designed to predict both the manner in which inputs are reflected in the system's outputs and the evolution of a system's internal state over time and in response to specific inputs [2].

3.2 Least squares method

The least squares method is a regression technique that is used to determine the line that best fits a given set of data. It minimizes the sum of the squared differences between the observed data and the corresponding values implied by the regression function. Each data point reflects the relationship between a known independent variable and an unknown dependent variable [8].

To enhance the model, we incorporated ridge regression (L2 regularization), which helps to reduce overfitting during model training [14].

3.3 Data Processing

For our project, we used 223 CSV files, each containing the data of a jump, measured on the flying hill of Gorišek brothers in Planica, Slovenia. Each contains 17 columns (`'Position'`, `'Height above ground'`, `'Time'`, `'X'`, `'Y'`, `'Z'`, `'Opening Angle'`, `'Stalling Angle Left'`, `'Stalling Angle Right'`, `'Roll Angle Left'`, `'Roll Angle Right'`, `'Yaw Angle Left'`, `'Yaw Angle Right'`, `'Speed hor.'`, `'Speed vert.'`, `'Speed resulting'`, `'WindTime|WindName|WindSpeed|Wind...'`) and the number of rows corresponding to the length of the jump. Data are recorded for every meter of air distance from the take-off point.

The data required some pre-processing before it could be used for training the model.
The column `WindTime|WindName|WindSpeed|...` combined multiple attributes separated by '|'. Data from 12 sensors, each measuring six wind characteristics, were expanded into $12 \times 6 = 72$ columns, one per sensor–feature pair (`sensor_feature`).

Position - air distance from the take-off point in meters. Begins with a negative value, which represents the distance from the starting point to the take-off point. In ski jumping, the starting point is adjusted according to the wind conditions, so this value is not constant.
Height above ground - height above ground in meters.
Time - time of the jump in seconds from the start of the jump.
X, Y, Z - coordinates of the jumper in a 3D space in meters. The X axis is aligned with the hill direction, the Y axis is across the hill, and the Z axis is vertical. The take-off point is $(0, 0, 0)$ as shown in Figure 1
Opening Angle - angle between the skis in degrees.
Stalling Angle Left, Stalling Angle Right - angle between the chord line of the left/right ski and the horizontal plane in degrees.
Roll Angle Left, Roll Angle Right - angle of the left/right ski around its longitudinal axis relative to the horizontal plane in degrees.
Yaw Angle Left, Yaw Angle Right - angle between the left/right ski and the horizontal plane in degrees. (angles are shown in Figure 2)
Speed hor., Speed vert., Speed resulting - horizontal, vertical, and the resulting speed of the athlete in km/h [15].

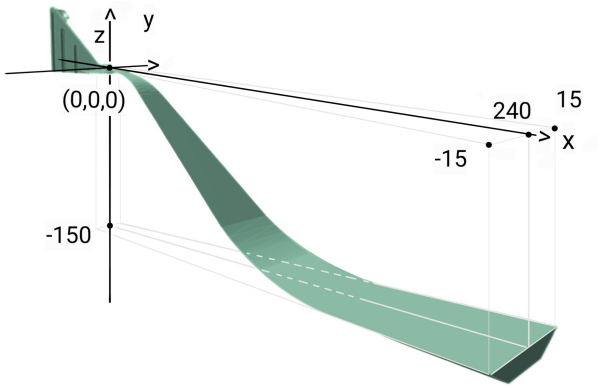

Figure 1: 3D model of Ski jump in Planica with added coordinates [12, 1]

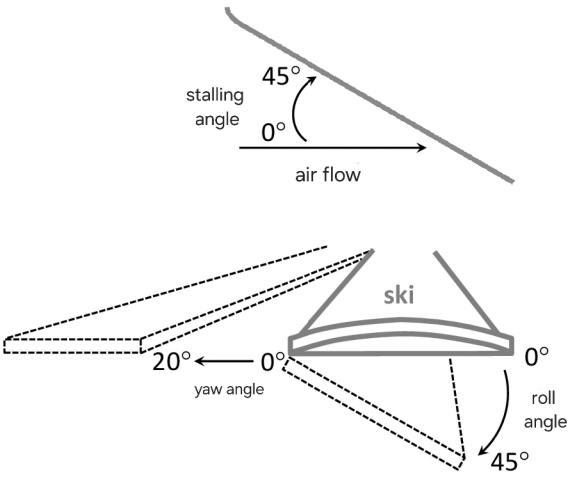

Figure 2: Different angles affecting the jump

The wind features are as follows:

WindTime - time of the wind measurement in the same format as the Time column itself. Since wind measurements are recorded less often, the wind values are applied to the most recent jump measurement and then just repeated until a new wind measurement is available. Since the wind

is represented by a nonlinear function, it would be hard to capture its movements with interpolation, so we decided to drop this column.

WindName - name of the sensor (Wi for $i = 1, \ldots, 12$)

WindSpeed - resulting speed of the wind in km/h

WindSpeedTangent - speed of the wind tangent measured along the x axis (hill direction) in km/h

WindTurbulence - vertical speed of the wind turbulence in km/h

WindSpeedCleanTan - wind speed tangent with turbulence removed in km/h

WindSpeedCross - speed of the wind measurement along the y axis across the hill in km/h

There are 12 wind sensors spread across the ski jump hill. To help with the analysis, we separated the jump section of the hill into 3 zones. The first zone contains wind sensors 1 to 4, the second zone contains sensors 5 to 8, and the third zone contains sensors 9 to 12 [13].

During processing, we also removed some ski jumps that were incomplete or had corrupted data, so the final dataset contained around 200 ski jumps.

4 Methodology

This section describes our research methodology. We first present different variations of the SSM that we tested for the ski jump simulation, followed by describing our model and how it predicts the jumps. Finally, we present the description of our ski jump animation app.

4.1 Different modeling approaches

In addition to pure SSM, we considered different approaches for modeling ski jumps that included classical physics-based models, but the data are not sufficient to accurately capture all the forces acting on the jumper. We also tried a hybrid approach that combined SSM and Physics-informed Neural Networks (PINNs [16]), where the SSM would provide a baseline prediction and the PINN would learn to correct any discrepancies, taking into account physical properties of the system, such as the mass of the pilot, the properties of the wind, and gravitational force [4]. These parameters are included in the equations of motion and added to the total loss function. So, the model prefers solutions that are consistent with the laws of physics. This turned out to be less effective than a pure SSM approach, but the reason exceeds the purpose of this paper. More about errors and models' comparison is given in Section 5.1.

4.2 Ski jump prediction model

In order to fit our data to the SSM, we stored the data in each file in three vectors. The main vector contains states or state variables of the system, which in our case are the X, Y, and Z coordinates, jumper velocities for each of the directions, resulting speed, and all angles (opening, stalling, roll, and yaw) [7].

The observation vector contains the measured outputs of the system, which in our case are the X, Y, and Z coordinates and height above ground. The controls contain the external inputs to the system, which in our case are the wind measurements from all the sensors for each of the features (speed, tangent, cross and turbulence) that are then averaged over each zone combined with all the angles (resulting speed, opening, and for both left and right ski the roll, yaw and stall angles).

We then used ridge regression to estimate the matrices A, B, C and D of the SSM, as shown in Figure 3, where we minimized the computed values from the current and previous values and the next time-stamped values. The next value is simulated through matrix multiplication with state, controls and observations vectors. For our simulation we specifically focus on A and B matrices, since we want to predict the next state from the current state and the current control. The C and D matrices are used to predict the next observation, but since we are interested in the trajectory, we don't use them for simulation. Thus, matrix A computes the next state from the current state, B computes the next state from the current control, C computes the next observation from the current state, and D computes the next observation from the current control. We then combine the predictions for next state and the next control to get the next simulated value, which allows us to then use recursion to get the full simulated jump. This allows us to predict the jump trajectory based on the environmental conditions and the starting state of the jumper [11].

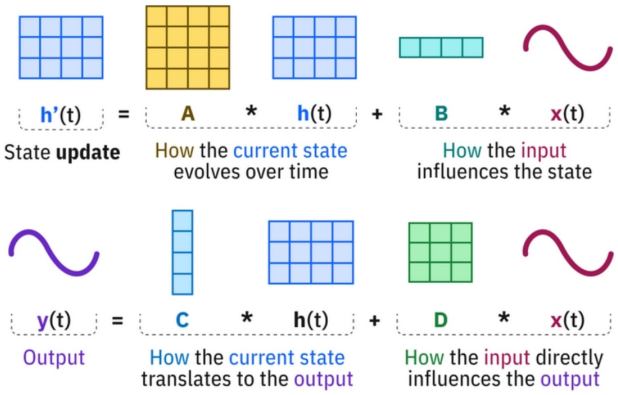

Figure 3: Schema of SSM matrices [3]

4.3 Average file starter

Since the goal is to have an interactive application where users can adjust the wind conditions and angles and observe the impact on the jump, we needed to find a way to start the simulation with the initial state of the jumper, that wouldn't be too specific to a single jump. We created an average file, where the values for each column are averaged over all the jumps in the dataset. Since the flights are of different lengths and measured at different time stamps, we first had to interpolate the jumps to get the average number of time stamps and the average length of the jump.

We then used the first row of the average file as the starting point for the simulation, which contains the average values for all the features at the take-off point. This allows the simulation to start with realistic initial conditions that are representative of the average jump in the dataset. And in case there is no external input from the user for a certain wind or angle, the simulation will just use the average values from the file as the default input, which allows the user to see a realistic jump trajectory even without adjusting the inputs.

We then calculated the matrices A and B using the same process as before on all of the jumps in the dataset. This allowed us to connect them with the app, so the user gets a realistic jump trajectory based on his inputs.

4.4 Ski jump animation app

To make our results accessible beyond the research setting, we developed an interactive web application using Shiny for Python [9]. The application serves as a front-end to the trained state-space model and allows users to explore ski jump simulations under varying environmental conditions or just to observe different measured ski jumps. Firstly, through a set of input controls, users can adjust factors such as wind speed, wind directions, or different ski angles, and the application instantly updates the predicted jump trajectory. Secondly, users can simply explore random jumps from the provided dataset or upload their own CSV file of measured jumps, as long as it includes the columns described in Section 3.3.

The application presents the results as an animated visualization of the ski jump, showing the full trajectory and the final distance. In this way, the application functions both as an analytical tool, helping to test how different conditions affect performance, and as an educational resource that makes the mechanics of ski jumping easier to understand for a wider audience. It is available online. [1]

5 Main results

In this section, we present the results of our simulations. Firstly, we present a statistical comparison of all the models, followed by a precise analysis of our predictions.

5.1 Models' error

In order to evaluate different models, we first had to define a metric to measure the prediction error. Since actual and simulated jumps are represented with x, y, and z coordinates but are measured at different time stamps and can contain a different number of measurements, we had to find a way to compare them. Since time stamps represent the athlete's position on a coordinate system at different times, we decided the time information isn't as crucial as the position and consequently the distance of the jump. Which allowed us to compare the actual and simulated jumps irrespective of the information about time.

We first tried to project the shorter trajectory on to the other one and compute the distance between the original and the projection, but this method turned out to be computationally expensive. So we decided to compute the distance between the actual and the simulated jumps by interpolating both jumps. The new measurements contain the start and end point and all the ones, where x reaches a natural value. We then compute the error as the norm of the difference between the two jumps. And after one of the jumps ends, we just add the distance from the end of the shorter jump to the end of the longer jump to the error. This way, we penalize the model for not being able to predict the correct length of the jump, while still keeping the interpretability of the error as the distance between the two jumps.

We then used a KFold cross-validation to evaluate the models. KFold is a method of splitting the dataset into K subsets, or folds, and then train the model K times, each time using a different fold as a test set and the remaining folds as a training set. Since we already had a very limited number of jumps, we used the leave-one-out cross-validation version of the algorithm, where K is equal to the number of jumps in the dataset. This allows us to use as much data as possible for training while still providing a robust evaluation of the model's performance on unseen data. We then computed the average error between the actual jump and the simulated jump for both the training set and the test set, as shown in Figure 4.

At the end it turned out that the best model was the pure SSM with averaged wind data over the zones, which with the leave-one-out algorithm achieved an average error of 1.32 m on the training data and 1.37 m on the test data. And when we use starting point from the average file instead of the actual starting point of the jump, the error is 1.58 m on the training data and 1.63 m on the test data. So we can interpret the error as: If we were to choose a random point on the simulated path, then the athlete would be on average 1.63 m away from the actual path of the jump.

This suggests that averaging the wind data over the zones helps to reduce noise and improve the model's ability to generalize from the training data to unseen jumps.

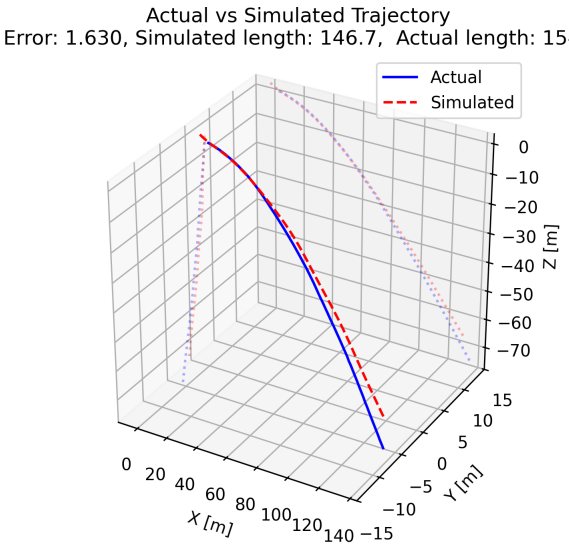

Figure 4: Actual vs. simulated ski jump trajectory

5.2 Analysis of our model

In the process of developing our ski jump prediction model, we evaluated several variations to determine the most effective approach. We compared the performance of a pure SSM with a hybrid model that combined SSM with PINN. The pure SSM demonstrated superior predictive accuracy, probably due to its ability to directly model the temporal dynamics of ski jumps without the added complexity of PINNs.

We tested the impact of wind features on the model's performance by training the models with the information of every wind sensor instead of averaging the wind data over zones. However, this approach resulted in a higher prediction error (with the error measuring 1.62 m on the training data and 1.82 m on the test data), likely due to the increased noise and variability in the data from the individual sensors, which may have made it more difficult for the model to learn the underlying patterns in the data.

Wind is a critical factor in ski jumping, so we attempted to capture its nonlinear effects by including columns for the squared wind features. However, we found that adding these squared terms did not help with the prediction, and in fact, it increased the error (with the error measuring 1.33 m on training data and

[1] https://camlekn.shinyapps.io/ski-jump/

1.43 m on test data). This suggests that the relationship between wind and jump trajectory may not be well captured by simple polynomial terms.

Here we present all the results of the different models in Table 1.

Model	Training Error	Test Error
SSM with averaged wind data	1.32 m	1.37 m
SSM with averaged wind data and average starting point	1.58 m	1.63 m
SSM with individual wind sensor data	1.62 m	1.82 m
SSM with averaged wind data and squared wind features	1.33 m	1.43 m

Table 1: Comparison of different models based on training and test error

Since the simulation still requires numerous inputs, we made it interactive, allowing users to adjust the wind conditions and observe their impact on the jump. In the ski jumping app, users can manipulate sliders to set the wind speed, wind tangent, wind cross, and wind turbulence for each of the three zones. As a result, the wind loses its original movement function in the simulation. All other inputs are set to the average values computed from the dataset.

5.3 Analysis of model's parameters

To better understand the model's behavior, we analyzed the parameters of the matrices A and B.

Matrix A suggests that the SSM is capturing two different physical regimes: slow kinematics and fast aerodynamic control. The strongest values are the diagonal terms for X, Y, and Z, which show that position depends overwhelmingly on its previous state. By contrast, posture variables are more reactive than persistent, especially Opening Angle, Roll, and Yaw, indicating active correction during the jump. The strongest non-diagonal dependency is $v_y \rightarrow$ Roll Left (0.302), followed by $v_y \rightarrow$ Roll Right (0.233), which implies that vertical motion is a major trigger for balance corrections. Another meaningful pattern is Opening Angle $\rightarrow$ Yaw Left/Right with opposite signs, suggesting that changing ski opening may also influence heading asymmetrically. In real-world terms, the jumper's path is smooth, but the body-ski system is continuously making small directional and balance adjustments to preserve lift and control.

Matrix B indicates that environmental conditions are concentrated in the control layer of the system. The strongest dependencies are not on position, but on `Opening Angle`, `Roll`, `Yaw`, and `Stalling`, meaning the athlete mainly responds to disturbances by changing posture. The dominant external source is a variable `landing_Turbulence`, which strongly pushes opening, roll, and stalling responses. This makes physical sense: wind or unstable air does not instantly move the jumper's whole trajectory, but first forces the body-ski system to react, and those reactions then propagate into flight behavior.

6 Discussion

This section examines the predictive performance of the trajectories, highlights the limitations of our current approach, and suggests directions for future improvements.

6.1 Limitations

Given the relatively small dataset of ski jumps, the main limitation of our model lies in the limited data available for training the model. After preprocessing, the dataset contained only about 200 jumps, which may limit the SSM's ability to represent the full range of trajectory variations under different circumstances. As a result, the model may struggle to accurately predict jumps under novel or extreme conditions.

Furthermore, the dataset lacks detailed information, or any information at all, about individual jumpers, such as body weight, sex, or other physiological characteristics that are known to influence jump performance. Incorporating these variables could improve model accuracy and provide more personalized predictions [6].

Lastly, due to limited computing power, only one CPU was available, restricting the use of possible better models. To address these challenges, using cloud-based resources could help run larger models and improve the prediction of trajectories.

6.2 Future work and potential improvements

Although the current approach shows promise, there are several avenues for future improvements. Some of which we are working on at the time of writing this paper.

Currently, we are working on improving the sliders' functions on the web application. Since the wind data determined by the user is static throughout the jump, this adds a lot of generalization. In reality, wind conditions can change rapidly during a jump. So we would like to add additional controls to the app that would allow the user to define how the wind changes during the jump. They could choose whether the wind would gain or lose a certain feature (such as speed or turbulence) during the jump.

Expanding the dataset to include more jumps and additional contextual information about individual jumpers could improve the accuracy of the model. We could try to generate more data by using data augmentation techniques, such as adding noise to the wind measurements or slightly modifying the angles. We could also try to find the nonlinear movements of the wind and interpolating the wind measurements by their original time stamps to better capture the wind dynamics.

7 Conclusion

This paper presents a method for predicting ski jump trajectories based on environmental conditions. By incorporating external factors into the modeling framework and applying least squares estimation, we demonstrated that the model is capable of capturing the dynamics of ski jumps and producing realistic trajectory predictions. In addition, we developed an interactive application that makes the results accessible to a broader audience through simulations and animations of predicted jumps. Although the current model is limited by the size of the dataset and the absence of certain athlete-specific variables, the results show that state-space models are a promising tool for analyzing ski jumping performance.

8 Acknowledgments

This work was supported by the Slovenian Research and Innovation Agency(ARIS) via the Research Program Knowledge Technologies(grant P2-0103) and the European Commission through the Horizon Europe projects SLAIF (grant 101254461), ELIAS (grant 101120237), and HumAlne (grant 101120218). We would

like to thank Smučarska Zveza Slovenije (Ski Association of Slovenia) for providing the ski jump data.

References

[1] 3DWarehouse via 3dmdb.com. 2025. "ski jumping planica" [3d model]. https://3dmdb.com/en/3d-model/ski-jumping-planica/8386000/?free=True&q=Ski+jump. Free model; accessed: 2025-08-26. (2025).

[2] Masanao Aoki. 1990. *State Space Modeling of Time Series*. (2nd, revised and enlarged ed.). *Universitext*. Springer Berlin Heidelberg, Berlin, Heidelberg. ISBN: 978-3-642-75883-6. doi:10.1007/978-3-642-75883-6.

[3] Dave Bergmann. 2025. What is a state space model? Accessed: 2025-09-24. https://www.ibm.com/think/topics/state-space-model.

[4] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, and George E. Karniadakis. 2021. Physics-informed neural networks (pinns) for fluid mechanics: a review. *Acta Mechanica Sinica*, 37, 12, 1727–1738. doi:10.1007/s10409-021-01148-1.

[5] Xiao Fang, Benedikt Grüter, Patrick Piprek, Valentina Bessone, Jens Petrat, and Florian Holzapfel. 2020. Ski jumping trajectory reconstruction using wearable sensors via extended Rauch-Tung-Striebel smoother with state constraints. *Sensors*, 20, 7, 1995. doi:10.3390/s20071995.

[6] Wolfram Müller. 2008. Performance factors in ski jumping. In *Sport Aerodynamics*. CISM International Centre for Mechanical Sciences. Vol. 506. Helge Nørstrud, editor. Online ISBN: 978-3-211-89297-8. Springer, Vienna, 139–160. ISBN: 978-3-211-89296-1. doi:10.1007/978-3-211-89297-8_8.

[7] Wolfram Müller. 2006. The physics of ski jumping. Tech. rep. CERN report on the aerodynamics and physics of ski jumping. CERN. https://cds.cern.ch/record/1009275/files/p269.pdf.

[8] Bor Plestenjak. 2016. *Razširjen uvod v numerične metode*. Slovenian textbook on numerical methods. DMFA-založništvo.

[9] Posit Team. 2025. Shiny for python. Accessed: 2025-08-29. https://shiny.posit.co/py/.

[10] Leo Schwinn, René Raab, Arne Nguyen, Dario Zanca, and Bjoern M. Eskofier. 2022. xLength: predicting expected ski jump length shortly after take-off using deep learning. *Sensors*, 22, 21, 8474. doi:10.3390/s22218474.

[11] Serrano.Academy. 2025. State-space model (ssm) tutorial. https://youtu.be/g1AqUhP00Do. State-Space Model (SSM) video. (2025).

[12] Ski Jumping Hill Archive, skisprungschanzen.com. 2025. Letalnica (letalnica bratov gorišek), planica, slovenia — ski jumping hill archive. https://www.skisprungschanzen.com/EN/Ski+Jumps/SLO-Slovenia/Planica/0475-Letalnica/. Accessed: 2025-09-12. (2025).

[13] Ava Thompson, ed. 2025. *Ski Jumping*. Found via Google Books at https://www.google.si/books/edition/Ski_Jumping/G2pPEQAAQBAJ?hl=en&gbpv=0. Publifye AS.

[14] Wessel N. van Wieringen. 2015. Lecture notes on ridge regression. arXiv preprint arXiv:1509.09169. Revision v8, submitted 30 September 2015; revised 27 June 2023. (2015). doi:10.48550/arXiv.1509.09169.

[15] Mikko Virmavirta and Juha Kivekäs. 2019. Aerodynamics of an isolated ski jumping ski. *Sports Engineering*, 22, 1, 1–6. doi:10.1007/s12283-019-0298-1.

[16] StatQuest with Josh Starmer. 2025. Neural networks tutorial. https://youtu.be/CqOfi41LfDw. Neural networks introduction video. (2025).

[17] Daniel Zecha, Christoph Eggert, Michael Einfalt, Stefan Brehm, and Rainer Lienhart. 2018. A convolutional sequence to sequence model for multimodal dynamics prediction in ski jumps. In *Proceedings of the 1st International Workshop on Multimedia Content Analysis in Sports*, 11–19.